

Please watch or skim the videos listed on the Canvas assignment corresponding to this LGA. There is a lot more content than usual in this LGA, since it should be review material for most of you. Feel free to fast-forward though any bits you feel pretty confident about already.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

- Using the operator `&`, we can obtain the address of the value stored in a variable. The address can be stored in a pointer type variable.

Note this is the same as the memory address where the variable is stored in memory.

- Using the operator `*`, known as the indirection (or dereferencing) operator, we can get or set the value pointed to by a pointer.
- Array variables are secretly pointers; however, describe one difference:

Array variables are different because you can't change the address they point to.

- To the compiler, `p[j]` is equivalent to `*(p + j)`.
- If `p` is an `int*`, and `p = 0x1000`, what is the value `p + 3`?
`0x1000 + 3 * sizeof(int)`; E.g., with 4-byte `int` values, the answer is `0x100C`.
- The memory segment containing the function call stack is generally called the stack.
- Dynamically allocated memory for C++ comes from the free store or heap.
- The new operator is used to dynamically allocate memory; the result of the operation is a pointer to the memory.
- The delete (for single objects or values) or delete[] (for arrays) operator is used to deallocate dynamically allocated memory.
- Name two types of pointer you should never try to dereference:
dangling pointer, uninitialized pointer, null pointer (nullptr).
- If your program regularly loses pointers to dynamically allocated memory, your program may have this kind of memory error: memory leak.

Group questions

Distribute the following questions across the members of your group. You will share your solutions (and most importantly the *method* of your solutions) during the next lecture period. Divide up the questions so that each question has at least three solutions from different group members. *Note:* from time to time we may include trick questions.

1. (Basic) Provide the missing line in the code below so that the program prints “42”:

```
void go(int* ptr) {
    // <- put your code here
}

int main() {
    int x = 17;
    go(&x);
    cout << x << endl;
    return 0;
}
```

```
*ptr = 42;
```

2. (Basic) What is the output of the following code?

```
int a = 42;
int b = 17;

int* p = nullptr;
int* q = &a;

*q = 99;

p = q;
q = &b;

*q = 22;
*p = 77;

cout << a << " " << b << " " << q << endl;
```

(after) c++ statement	a	b	p	q
int a = 42;	42	?	?	?
int b = 17;	42	17	?	?
int* p = nullptr;	42	17	null	?
int* q = &a;	42	17	null	&a
*q=99;	99	17	null	&a
p=q;	99	17	&a	&a
q=&b;	99	17	&a	&b
*q=22;	99	22	&a	&b
*p=77;	77	22	&a	&b

output: 77 22 0xdeadbeef

(if (deadbeef)₁₆ happens to be the address of variable b. Expect your system to print out some other number for &b. 0xdeadbeef is only one of many favored “[Magic Debug Numbers](#)” commonly used by programmers.)

3. (Intermediate) What is the output of the following code?

```
int upper = 10;
int arr[11];

for (int i = 0; i <= upper; i++) arr[i] = i;

for (int* p = arr; p <= arr + upper; p = p + 3) {
    cout << *p << " ";
}
cout << endl;

for (int* q = arr + upper; q >= arr; q = q - 2) {
    cout << *q << " ";
}
cout << endl;
```

The first for-loop fills `arr` with its index values, the second loop prints every third one:

0 3 6 9

The last loop prints every second value but begins at the last element in the array and works backwards to the beginning:

10 8 6 4 2 0

4. (Intermediate) Given some class `foo` with a public method named `run()`, write the function `run_all()` that takes in an array of `foo` objects and the length of the array and calls the `run()` method on each object. Give *two* different solutions for looping over the array (i.e., one using array indexing and another using pointers):

```
class foo {
public:
    void run(); // actual behavior defined elsewhere
};
```

```
void run_all(foo* objects, int len) {
```

```
}
```

```
void run_all(foo* objects, int len) {
    for( int i = 0; i<len; i++ ) {
        objects[i].run();
    }
}

void run_all(foo* objects, int len) {
    for (foo* p = objects; p < objects + len; p++) p->run();
}

void run_all(foo* objects, int len) {
    for( int i = 0; i<len; i++ ) {
        (*(objects + i)).run();
    }
}

void run_all(foo* objects, int len) {
    foo* end = objects + len;
    while( objects < end ) (objects++)->run();
}
```

You can't compile all of these solutions in into one executable (do you see why?), but each is independently a correct response.

5. (Basic) Write a program to dynamically allocate an array of 80 `ints`, fill every array position with 17, then deallocate the array.

```
int *a = new int[80];
for (int i = 0; i < 80; i++) a[i] = 17;
delete[] a;
return 0;
```

6. (Basic) What is problematic with this code?

```
double* get_sum(double x, double y) {
    double ans = x + y;
    return &ans;
}
```

The code returns an address to a *local* (i.e., stack allocated) variable in a function, which will be deallocated after the function exits. In other words, the return value will be pointing to a memory that is returned to the stack and free to be used by some other function calls; using the pointer (dereferencing it) is a very bad idea (and may crash your program).

7. (Basic) What is problematic with this code?

```
int main() {
    char* s;
    while (true) {
        int n = rand() % 10000;
        delete[] s;
        s = new char[n];
        for (int i = 0; i < n; i++) s[i] = 'a';
    }
}
```

The variable `s` is not initialized to any valid pointer value and therefore the first delete operation will almost certainly try to deallocate memory that is not dynamically allocated. One quick fix would be to initialize `s` to `nullptr`, since `delete[]` can be applied to `nullptr` without harm. (The code above may run fine on your system; some operating system/compiler combinations are more forgiving than others, and even just by luck it is possible that `s` will start with value 0 (which is the usual value assigned to `nullptr`).)

8. (Intermediate) Explain the bug or issue with the code below, or provide a convincing argument that none exists:

```
void release(char* p, int n) {
    if (n % 2 == 0) delete[] p;
}

int main() {
    int a = rand();
    char *x = new char[100];
    if (a % 2 == 1) delete[] x;
    release(x, a);
}
```

The pointer variable `x` is initialized to point to an array of 100 chars and the delete operation will be executed either in the main function or in the release function, but not both, because a random integer can either be odd or even and it cannot be both. This code is technically bug-free, although very poorly designed.

9. (Intermediate) Explain the bug or issue with the code below, or providing a convincing argument that none exists:

```
int* generate(int n, int bound) {
    int* arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % bound;
    }
    return arr;
}

int main() {
    int* result;
    for (int x = 1; x <= 100000; x++) {
        result = generate(x, x);
        for (int i = 0; i < x; i++) {
            cout << result[i] << " ";
        }
        cout << endl;
    }
    delete[] result;
    return 0;
}
```

Because `delete[] result` is outside the for loop, every iteration of the for loop except the last allocates an array that is never deallocated. Because of this memory leak, we will most likely run out of memory and get a runtime error.